

Module 1: Spring Framework, Spring Boot, and Maven

Questions

Q1. What is the Spring Framework?

Answer: Spring is a lightweight Java framework for building enterprise applications. It gives you a container that manages objects called beans and connects them through dependency injection. In a banking system, Spring can manage your `AccountService`, `TransactionRepository`, and `PaymentGateway` objects without you manually creating them with `new`.

Q2. What are the advantages of the Spring Framework?

Answer: Spring reduces boilerplate code, supports transactions, integrates with databases and messaging, and makes unit testing easy through dependency injection. For example, in a supply chain application, you can swap a real `ShippingClient` with a mock during tests because Spring injects the dependency from outside.

Q3. What are the main modules of the Spring Framework?

Answer: Core modules include Spring Core (beans and DI), Spring AOP (cross-cutting concerns), Spring Data Access (JDBC, JPA), Spring MVC/Web (REST and web apps), Spring Security (authentication and authorization), and Spring Test. A banking ledger app might use Core, Data Access, and Security together.

Q4. Why is Spring called a lightweight framework?

Answer: Spring does not force you to use heavy application servers or inherit from specific classes. You use plain Java objects called POJOs and configure them with annotations or XML. This makes a payment microservice small and fast to start.

Q5. What is Inversion of Control (IoC) in Spring?

Answer: Inversion of Control means the Spring container controls object creation and wiring instead of your code doing it. You declare what you need, and Spring provides it. In a supply chain order service, instead of the service creating a `WarehouseClient`, Spring injects a ready-made one.

Q6. What are the ways to create a Spring Boot project?

Answer: You can use Spring Initializr (start.spring.io) in a browser, the Spring Boot CLI, IDE plugins in IntelliJ IDEA or Eclipse, or manually create a Maven/Gradle project. In a banking team, developers usually use Spring Initializr to generate a project with `spring-boot-starter-web` and `spring-boot-starter-data-jpa`.

Q7. What is Spring Initializr?

Answer: Spring Initializr is a web tool that generates a ready-to-import Spring Boot project. You pick the language, build tool, Spring Boot version, and dependencies, and it creates the project

structure, `pom.xml`, and main class. It saves time when starting a new payment API service.

Q8. What is the typical structure of a Spring Boot project?

Answer: A typical project has `src/main/java` for code, `src/main/resources` for config and static files, `src/test/java` for tests, a main class with `@SpringBootApplication`, and `application.properties` or `application.yml`. In a supply chain app, packages are often `controller`, `service`, `repository`, `model`, and `config`.

Q9. What is the role of the main class in a Spring Boot application?

Answer: The main class contains the `main` method and is annotated with `@SpringBootApplication`. Calling `SpringApplication.run()` starts the Spring context, triggers auto-configuration, begins component scanning, and starts the embedded server. It is the single entry point for a banking microservice.

Q10. What is the default embedded server in Spring Boot?

Answer: The default embedded server is Tomcat. Spring Boot also supports Jetty and Undertow. You can switch by excluding Tomcat and adding the Jetty starter. This is useful in high-throughput supply chain APIs where Jetty's threading model may perform better.

Q11. What is a Spring Bean?

Answer: A Spring Bean is any Java object that is created, configured, and managed by the Spring IoC container. Beans are usually annotated with `@Component`, `@Service`, `@Repository`, or `@Controller`. In a banking app, a `FundTransferService` is a bean that Spring injects wherever it is needed.

Q12. What are Spring bean scopes?

Answer: The common scopes are singleton (one instance per context), prototype (new instance on every request), request (one per HTTP request), and session (one per HTTP session). Singleton is the default. A supply chain `CurrencyConverter` might be singleton, while a per-request `OrderBuilder` could be prototype.

Q13. What is the default bean scope in Spring?

Answer: The default scope is singleton. Spring creates exactly one instance of the bean and shares it across the entire application context. This is efficient for stateless services, such as a banking `InterestCalculationService`.

Q14. Are singleton Spring beans thread-safe?

Answer: No, singleton beans are not automatically thread-safe just because Spring creates one instance. If the bean holds mutable shared state, multiple threads can corrupt it. You should make beans stateless or use synchronization. A banking `AccountBalanceCache` must use thread-safe collections or locks.

Q15. What is the Spring bean lifecycle?

Answer: Spring creates the bean, injects dependencies, runs initialization callbacks like `@PostConstruct`, makes it available for use, and finally calls destruction callbacks like `@PreDestroy` when the context closes. In a supply chain app, you might use `@PostConstruct` to load region codes into memory on startup.

Q16. What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?

Answer: All four create Spring beans, but they signal different layers. `@Component` is generic. `@Service` marks business logic. `@Repository` marks data access and enables exception translation. `@Controller` marks web controllers. A banking app uses `@Service` for `PaymentService` and `@Repository` for `TransactionRepository`.

Q17. What is the difference between `@Bean` and `@Component`?

Answer: `@Component` is used on classes that Spring scans automatically. `@Bean` is used on methods inside a `@Configuration` class to create beans manually, often for third-party objects you cannot annotate. For example, you might define a `@Bean` for a banking SDK client that is not under your control.

Q18. What is Dependency Injection (DI)?

Answer: Dependency Injection means an object receives its dependencies from outside instead of creating them itself. Spring provides the dependencies at runtime. In a supply chain app, `OrderService` does not create `InventoryClient`; Spring injects it, so the service is easier to test and change.

Q19. What are the types of dependency injection in Spring?

Answer: Spring supports constructor injection, setter injection, and field injection. Constructor injection is recommended because it makes dependencies required and immutable. Field injection is easy but hides dependencies. In a banking `LoanApprovalService`, constructor injection ensures all required services are present at startup.

Q20. Why is constructor injection preferred in Spring?

Answer: Constructor injection makes dependencies explicit, allows immutable objects, makes unit testing easy, and prevents a bean from being created with missing dependencies. It also avoids the circular dependency problems that constructor injection itself detects at startup.

Q21. What is `@Autowired`?

Answer: `@Autowired` tells Spring to inject a matching bean automatically by type. It can be placed on a constructor, setter, or field. If multiple beans match, you can resolve the conflict with `@Qualifier` or `@Primary`. In a banking app, `@Autowired` injects a `NotificationSender` into a `PaymentService`.

Q22. What is the difference between @Qualifier and @Primary?

Answer: @Primary marks one bean as the default choice when multiple beans of the same type exist. @Qualifier is used at the injection point to pick a specific bean by name. If you have both EmailNotificationSender and SmsNotificationSender, you can mark one @Primary or use @Qualifier("email").

Q23. What is a circular dependency in Spring, and how do you resolve it?

Answer: A circular dependency happens when Bean A depends on Bean B and Bean B depends on Bean A. Spring fails to start with constructor injection. You can fix it by redesigning the classes, using setter or field injection, or using @Lazy. In a banking app, separating a PaymentService and AuditService via events can break the cycle.

Q24. What is the difference between IoC and DI?

Answer: Inversion of Control is the principle where the framework manages object creation and flow. Dependency Injection is one way to implement IoC by supplying dependencies from outside. So DI is the mechanism, and IoC is the broader design goal behind it.

Q25. What is the difference between Spring and Spring Boot?

Answer: Spring is the core framework that provides IoC, transactions, data access, and web support. Spring Boot is built on top of Spring and removes boilerplate by providing auto-configuration, embedded servers, and starter dependencies. In simple words, Spring gives you the engine, and Spring Boot gives you a ready-to-drive car.

Q26. Why do we use Spring Boot over Spring Framework?

Answer: Spring Boot reduces setup time with auto-configuration, starter POMs, embedded servers, and production-ready monitoring through Actuator. It lets a banking team create a working REST API in minutes instead of spending hours on XML configuration.

Q27. What are Spring Boot starter dependencies?

Answer: Starters are convenient dependency bundles. spring-boot-starter-web includes Spring MVC, Tomcat, and Jackson. spring-boot-starter-data-jpa includes Hibernate, Spring Data JPA, and a connection pool. For a supply chain app, adding one starter pulls in everything needed for a layer.

Q28. Does Spring Boot replace Spring Framework?

Answer: No, Spring Boot does not replace Spring. It makes Spring easier to use by providing defaults and conventions. Underneath, Spring Boot still uses Spring Core, Spring MVC, Spring Data, and other Spring modules.

Q29. When would you choose plain Spring over Spring Boot?

Answer: You might choose plain Spring when you need full control over every configuration detail, when you deploy to an external application server as a WAR, or when integrating with a

legacy codebase that already has custom Spring setup. Most modern banking microservices, however, use Spring Boot.

Q30. What is Spring Boot auto-configuration?

Answer: Auto-configuration automatically sets up Spring beans based on the dependencies present in your classpath. If `spring-boot-starter-data-jpa` is present, Spring Boot configures a `DataSource` and `EntityManagerFactory` for you. You can override defaults in `application.properties`. In a banking app, this means the database connection is ready without manual bean definitions.

Q31. How does Spring Boot auto-configuration work internally?

Answer: Spring Boot reads `META-INF/spring.factories` or `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` to find auto-configuration classes. Each class is decorated with `@Conditional` annotations, such as `@ConditionalOnClass` and `@ConditionalOnMissingBean`, so beans are only created when appropriate. If your supply chain app adds Redis, the Redis auto-configuration kicks in.

Q32. What is @SpringBootApplication?

Answer: `@SpringBootApplication` is a convenience annotation that combines `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`. It marks the main class, enables auto-configuration, and tells Spring where to look for components. Without it, you would need three separate annotations.

Q33. What is the ApplicationContext in Spring?

Answer: The `ApplicationContext` is the Spring IoC container. It holds bean definitions, creates beans, wires dependencies, and provides services like event publishing and internationalization. When a banking application starts, `SpringApplication.run()` creates the `ApplicationContext`.

Q34. What is the difference between ApplicationContext and BeanFactory?

Answer: `BeanFactory` is the basic container with lazy initialization and minimal features. `ApplicationContext` extends it with eager initialization, event propagation, AOP support, and easier integration. Modern Spring Boot apps always use `ApplicationContext`.

Q35. What happens when you run SpringApplication.run()?

Answer: Spring Boot creates the application context, loads property sources, runs `ApplicationRunner` and `CommandLineRunner` beans, starts the embedded web server, and enables auto-configuration. By the end, your banking payment API is listening on its configured port.

Q36. What is component scanning?

Answer: Component scanning is how Spring automatically discovers beans in specified packages. `@ComponentScan` tells Spring to look for `@Component`, `@Service`, `@Repository`, and `@Controller` annotations. In a supply chain app, Spring scans `com.company.supplychain.*` and registers all discovered beans.

Q37. How do you disable a specific auto-configuration class?

Answer: You can use `@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})` or set `spring.autoconfigure.exclude` in `application.properties`. This is useful when a banking app needs to turn off default datasource setup because it uses a custom connection factory.

Q38. What are `@Conditional` annotations in Spring Boot?

Answer: Conditional annotations control when beans are created. `@ConditionalOnClass` creates a bean only if a class is on the classpath. `@ConditionalOnProperty` creates it only if a property matches. `@ConditionalOnMissingBean` creates it only if the bean is not already defined. These make auto-configuration smart and safe.

Q39. How can you view all beans loaded by Spring Boot?

Answer: Inject the `ApplicationContext` and call `getBeanDefinitionNames()`. Spring Boot Actuator also provides the `/beans` endpoint. This helps during debugging when a banking app fails to start due to a missing bean.

Q40. What is Maven and what problem does it solve?

Answer: Maven is a build automation and dependency management tool for Java. It standardizes project structure, downloads required libraries from repositories, and provides lifecycle phases like compile, test, package, and deploy. In a banking project, Maven ensures every developer uses the same Spring Boot version.

Q41. What is a `pom.xml` file?

Answer: `pom.xml` is the Project Object Model file. It contains project metadata, dependencies, plugins, build settings, and parent POM references. It is the recipe Maven follows to build the project. A supply chain microservice `pom.xml` lists starters like `spring-boot-starter-web` and `spring-boot-starter-test`.

Q42. What are the Maven lifecycle phases?

Answer: The default lifecycle includes `validate`, `compile`, `test`, `package`, `verify`, `install`, and `deploy`. The clean lifecycle removes generated files. Running `mvn clean install` cleans old output, compiles code, runs tests, packages a JAR, and installs it in the local repository.

Q43. What is the difference between `mvn clean` and `mvn install`?

Answer: `mvn clean` deletes the `target` directory from previous builds. `mvn install` compiles, tests, packages, and copies the artifact to your local Maven repository. In a banking CI pipeline, you usually run `mvn clean install` to get a fresh, tested artifact.

Q44. What is the difference between compile-scope and test-scope dependencies?

Answer: Compile-scope dependencies are needed to compile and run the application, such as `spring-boot-starter-web`. Test-scope dependencies are only needed during testing, such as JUnit and Mockito. They are not included in the final production artifact.

Q45. What is the Maven local repository?

Answer: The local repository is a folder on your machine, usually `~/.m2/repository`, where Maven stores downloaded jars. When a teammate in a banking team adds a new dependency, Maven first checks this local cache before downloading from the internet or a corporate Nexus.

Q46. What is a Maven repository manager like Nexus or Artifactory?

Answer: A repository manager is a private server that stores internal artifacts and proxies public repositories. Banks use Nexus or Artifactory to share private libraries across teams, control versions, and improve build speed by caching external dependencies.

Q47. What is `dependencyManagement` in Maven?

Answer: `dependencyManagement` declares dependency versions centrally without adding them to the project directly. Child modules or dependencies can then omit the version. Spring Boot's parent POM uses this to align all starter versions automatically.

Q48. How do you exclude a transitive dependency in Maven?

Answer: You add an `<exclusions>` block inside the `<dependency>` tag. For example, if a banking library pulls in an old version of Jackson that conflicts with Spring Boot, you exclude it so Spring Boot's managed version is used.

Q49. How do Maven profiles help in a Spring Boot project?

Answer: Profiles let you define environment-specific configuration within the same POM. You can have `dev`, `test`, and `prod` profiles that activate different databases or logging levels. A supply chain team might activate a profile for local H2 testing versus production Oracle.

Q50. What is `spring-boot-starter-parent`?

Answer: It is a parent POM that provides default Java version, default Spring Boot dependency versions, plugin configurations, and resource filtering. Inheriting from it simplifies your `pom.xml` because you do not need to specify versions for every starter.

Q51. How do you run a Spring Boot application using Maven?

Answer: You can use `mvn spring-boot:run` to start the application directly from Maven, or build with `mvn clean package` and then run the generated JAR with `java -jar target/app.jar`. In a banking deployment pipeline, the JAR is built once and deployed to multiple environments.

Scenario-Based Questions

S1. Your team needs to build a new real-time fund-transfer microservice. How would you set it up using Spring Boot?

Answer: I would use Spring Initializr with Java, Maven, and dependencies `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`, and `spring-boot-starter-validation`. I would structure packages into `controller`, `service`, `repository`, `model`, and `config`. The main class would use `@SpringBootApplication` to enable auto-configuration and component scanning. This gives a working REST service with database and security support in minutes.

S2. The fund-transfer service needs a `TransactionService` and a `TransactionRepository`. How do you wire them using Spring DI?

Answer: I annotate `TransactionService` with `@Service` and `TransactionRepository` with `@Repository`. I inject the repository into the service using constructor injection and `@Autowired`. Spring creates both beans and wires them automatically. This keeps `TransactionService` testable because I can replace the real repository with a Mockito mock during unit tests.

S3. A developer accidentally used field injection everywhere. What risk does this create, and how do you fix it?

Answer: Field injection hides dependencies, making them hard to unit test without reflection, and it can mask circular dependencies until runtime. The fix is to move dependencies to a constructor and use constructor injection. Constructor injection makes dependencies explicit, supports immutable fields, and fails fast at startup if a dependency is missing.

S4. During load testing, the `TransactionService` singleton shows race conditions. What could be wrong?

Answer: Singleton beans are not thread-safe by default. If `TransactionService` holds mutable shared state, such as a counter or a non-thread-safe map, concurrent fund-transfer requests can corrupt it. The fix is to keep the bean stateless or use thread-safe data structures like `ConcurrentHashMap`. Better still, move per-request data into method parameters or use the prototype scope if the bean must hold request state.

S5. The service must support multiple environments: dev, UAT, and prod. How do you manage this in Spring Boot?

Answer: I use Spring profiles. I create `application-dev.yml`, `application-uat.yml`, and `application-prod.yml`. Each profile defines its own database URL, logging level, and feature flags. I activate the profile using `spring.profiles.active` in the environment or as a JVM argument. In prod, I also secure Actuator endpoints and disable dev tools.

S6. A warehouse order API receives a request, calls inventory to reserve stock, then calls payment to charge the customer. How do you design this in Spring Boot?

Answer: I create an `OrderController` annotated with `@RestController`, an `OrderService` annotated with `@Service`, and `InventoryClient` and `PaymentClient` beans. The controller maps the incoming request, the service orchestrates the steps, and the clients handle external calls. I inject all clients into the service via constructor injection so the flow is clear and testable.

S7. The order service sometimes fails because the inventory and payment clients are coupled. How do you break the circular dependency?

Answer: If `InventoryClient` depends on `PaymentClient` and vice versa, Spring fails at startup with constructor injection. I redesign the code so both clients are independent and the `OrderService` coordinates them. Alternatively, I can use `@Lazy` or setter injection as a short-term fix, but the real solution is better separation of concerns.

S8. The team adds both Tomcat and Jetty starters by mistake. How does Spring Boot decide which server to use?

Answer: Spring Boot checks classpath order and defaults to the first embedded server it finds, usually Tomcat if it appears first. To switch to Jetty intentionally, I would exclude the Tomcat starter from `spring-boot-starter-web` and add the Jetty starter explicitly. This controls which server runs the supply-chain API.

S9. You need to expose order health status to the operations team. What Spring Boot feature helps?

Answer: Spring Boot Actuator provides production-ready endpoints such as `/actuator/health`. I add the `spring-boot-starter-actuator` dependency and configure security so only authorized users can access sensitive endpoints. I can also create a custom health indicator that checks connectivity to the warehouse database and external payment gateway.

S10. The application does not start, and the error points to a missing bean. How do you investigate?

Answer: I check that the missing bean class has a Spring stereotype annotation such as `@Component`, `@Service`, or `@Repository`, and that its package is under the component scan

path defined by `@SpringBootApplication`. I also verify that any third-party bean is declared with `@Bean` in a `@Configuration` class. If needed, I list all loaded beans using `ApplicationContext.getBeanDefinitionNames()` or the Actuator `/beans` endpoint.

S11. A banking build passes locally but fails in the CI pipeline with dependency conflicts. How do you fix this with Maven?

Answer: I run `mvn dependency:tree` to inspect transitive dependencies and identify the conflicting versions. I then use the `dependencyManagement` section or the `<exclusions>` tag to force the correct version. If the team uses a repository manager like Nexus or Artifactory, I verify that the CI environment points to the same repository as local machines.

S12. The CI pipeline needs to build a JAR once and deploy it to dev, UAT, and prod. How do you configure Maven for this?

Answer: I run `mvn clean package` to produce a single JAR. Environment-specific configuration is externalized into profile-specific property files or environment variables, not baked into the JAR. Maven profiles can help run environment-specific tests, but the same artifact is promoted across environments to guarantee consistency.

S13. A developer checked in a change that breaks the Maven build for everyone. What Maven command helps isolate the problem?

Answer: I use `mvn clean install -X` to run the build in debug mode and capture detailed logs. I also use `mvn dependency:tree` to spot new conflicts and `mvn test` to identify failing tests. Once I find the problematic commit, I either revert it or work with the developer to fix the POM and dependency versions.

S14. During peak season, the order API must handle ten times the normal traffic. Which Spring Boot settings matter?

Answer: I tune the embedded server thread pool, configure database connection pool limits in `application.yml`, add caching with Redis or Caffeine, and consider scaling horizontally behind a load balancer. I also monitor Actuator metrics for response time and error rate. If the bottleneck is database connections, I increase the HikariCP pool size within safe limits.

S15. The order API must switch from an in-memory database to Oracle in production without code changes. How is this possible?

Answer: I externalize the datasource configuration into `application.yml` using placeholders or Spring profiles. In dev, the profile uses H2; in prod, the profile uses Oracle URL, username, password, and driver. Auto-configuration detects the database driver on the classpath and creates the datasource. No code changes are needed because the same JPA code works against both databases.

S16. A new starter was added, but Spring Boot now auto-configures a bean the team does not want. How do you disable it?

Answer: I disable the unwanted auto-configuration class using

`@SpringBootApplication(exclude = {UnwantedAutoConfiguration.class})` or by listing it in `spring.autoconfigure.exclude` in `application.yml`. This is useful when a starter pulls in a feature, such as a default datasource or security filter, that conflicts with custom configuration.

S17. You have three classes — `PaymentController`, `PaymentService`, and `PaymentRepository`. Annotate them.

Answer: `PaymentController` gets `@RestController` because it exposes REST endpoints. `PaymentService` gets `@Service` because it holds business logic. `PaymentRepository` gets `@Repository` because it accesses the database. I inject `PaymentService` into the controller and `PaymentRepository` into the service, all via constructor injection, so Spring manages the object graph.

S18. Two beans of type `NotificationSender` exist — one for email, one for SMS. How do you inject the correct one?

Answer: I use `@Qualifier("emailNotificationSender")` at the injection point to pick the email bean, or `@Qualifier("smsNotificationSender")` for SMS. Alternatively, I mark one bean `@Primary` if it should be the default. This avoids ambiguity when Spring finds multiple candidates of the same type.

S19. A bean must load exchange rates once at startup. Which lifecycle callback do you use?

Answer: I use `@PostConstruct` on a method inside the bean. Spring calls this method after dependency injection is complete but before the bean is used. I can call an external FX-rate service or load rates from a file. The bean stays a singleton and uses the loaded rates for all subsequent payment calculations.

S20. A build artifact is 200 MB because it includes unnecessary dependencies. What do you check?

Answer: I run `mvn dependency:analyze` and `mvn dependency:tree` to find unused or oversized transitive dependencies. I remove unused direct dependencies and add `<exclusions>` for large transitive libraries that the project does not need. I also verify that test-scoped dependencies are not accidentally marked as compile scope.